

INSTRUCTIONS

- Answer the questions in the spaces provided on the question sheets.
- Extra sheets may be used for rough work, but make sure to write your name and ERP on them. Answers on extra sheets will not be graded.
- Understanding the questions is part of the exam; do not ask for clarifications during the exam.

NAME

ERP

Question:	Tracing	Short Questions	Debugging	Coding	TOTAL
Marks:	20	25	20	35	100
Score:					

Question 1: Tracing [20 MARKS]

- (a) [6 MARKS] A robot moves along the x -axis. Its movement is described by the recursive function, `moveRobot`, which returns the robot's final x -coordinate after a given number of steps.

```
int moveRobot(int steps) {
    if (steps == 0) return 0;
    if (steps == 1) return 1;
    if (steps % 2 == 0) {
        return moveRobot(steps - 1) - 2;
    } else {
        return moveRobot(steps - 2) + 3;
    }
}
```

Given that the robot always starts at an initial position of $x = 0$, find its final x -coordinate for:

i. `moveRobot(2)?`

Solution: `moveRobot(2) → moveRobot(1) - 2 → 1 - 2 → -1`

ii. `moveRobot(3)?`

Solution: `moveRobot(3) → moveRobot(1) + 3 → 1 + 3 → 4`

iii. `moveRobot(5)?`

Solution: `moveRobot(5) → moveRobot(3) + 3 → 4 + 3 → 7`

(b) [14 MARKS] For each of the following code snippets, determine the exact output produced. Assume all necessary headers (e.g., `<iostream>`, `<vector>`, `<string>`, `<cctype>`) are included. Each snippet is independent.

i.

```
void modify(std::vector<int>& vec) {
    if (!vec.empty()) {
        vec.push_back(vec[0] + 5);
        if (vec.size() > 1) {
            vec[1] = 0;
        }
    }
}

int main() {
    std::vector<int> v = {10, 20, 30};
    modify(v);
    for (int x : v) {
        std::cout << x << " ";
    }
}
```

Solution: 10 0 30 15

ii.

```
int main() {
    int arr[] = {5, 10, 15, 20, 25};
    int* p1 = arr + 1;
    int* p2 = &arr[3];
    std::cout << *p1 << " ";
```

```

    p1++;
    std::cout << *(p2 - 1) << " ";
    std::cout << *p1 << " ";
    std::cout << arr[0];
}

```

Solution: 10 15 15 5

iii.

```

bool check0(int& value) {
    value += 1;
    return false;
}
bool check1(int& value) {
    value += 10;
    return true;
}
int main() {
    int x = 0;
    if (check0(x) && check1(x)) {}
    else if (check1(x) || check0(x)) {}
    std::cout << x;
}

```

Solution: 11

iv.

```

int main() {
    int num1 = 13, den1 = 4;
    double num2 = 13.0, den2 = 4.0;
    std::cout << (num1 / den1) << " ";
    std::cout << (num1 % den1) << " ";
    std::cout << (num2 / den2) << " ";
    std::cout << (int)(num2 / den1);
}

```

Solution: 3 1 3.25 3

v. **int** main() {

```
int total = 0;
for (int k = 0; k < 7; ++k) {
    if (k % 3 == 0) {
        continue;
    }
    total += k;
}
std::cout << total;
}
```

Solution: 12

vi. **struct Coordinates** {
 int latitude;
 int longitude;
};
int main() {
 Coordinates location = {100, 200};
 Coordinates* pointer = &location;
 pointer->latitude = pointer->latitude / 2;
 pointer->longitude -= 50;
 std::cout << location.latitude << "," << pointer->longitude;
}

Solution: 50,150

vii. **int** main() {
 std::string text = "Exam Time!";
 int e = 0;
 for (**char** character : text) {
 if (character == 'e') {
 e++;
 }
 }
 std::cout << e;
}

Solution: 1

Question 2: Short Questions [25 MARKS]

- (a) [6 MARKS] *Valid Declarations.* For each declaration or initialization below, check the circle if it is syntactically valid and would compile without errors.

☒ `int numbers[5] = {1, 2, 3};`

☒ `double* piPointer = new double(3.14);`

☐ `char name[];`

☒ `const int MAX = 100;`

☐ `int* data = {10, 20, 30};`

☒ `int x = 10;`

☒ `string message = "Hello";`

☒ `std::string city = "Karachi";`

- (b) [5 MARKS] *Scope & Lifetime.* Consider the following code:

```
#include <iostream>

int globalVariable = 10;

void update(int value) {
    value = value * 2;
    globalVariable += value;
}

int main() {
    int localVariable = 5;
    globalVariable += localVariable;
    update(localVariable);
    std::cout << "Local: " << localVariable << std::endl;
    std::cout << "Global: " << globalVariable << std::endl;
    return 0;
}
```

Which of the following statements are true regarding the execution of this code?

- ☐ The final value of `localVariable` printed will be 10.

- ✓ The final value of `localVariable` printed will be 5.
- The final value of `globalVariable` printed will be 20.
- ✓ The final value of `globalVariable` printed will be 25.
- The function update directly modifies `localVariable` in `main`.

(c) [8 MARKS] *Expression Evaluation*. Given the following code setup:

```
#include <string>

struct Book {
    std::string title;
    int pages;
    double price;
};

int main() {
    int count = 50;
    int* countPointer = &count;
    Book b1 = {"Programming Fun", 200, 25.50};
    Book* bookPointer = new Book{"Data Tales", 150, 19.99};
    // ... expressions below would use these ...
    return 0;
}
```

Determine the **type** and **value** of each expression below. If an expression would cause a compile-time error or undefined behavior at runtime, write Error.

Expression	Type	Value
*countPointer + 10		
b1.pages / 2		
bookPointer->title.length()		
b1.price - bookPointer->price		
countPointer == &count		
&b1 == bookPointer		
bookPointer[0].pages		
(count > bookPointer->pages)		

(d) [6 MARKS] *Array & Pointer Operations*. Consider the following code:

```
#include <iostream>
#include <vector>
int main() {
    std::vector<int> data = {10, 20, 30, 40, 50};
    int* p = &data[1];
    int* q = &data[3];

    *p = *p + 5;
    q = p;
    *(q - 1) = 99;
    *(p + 2) = *q + data[0];

    for (int value : data) {
        std::cout << value << " ";
    }
}
```

```
    }  
    std::cout << std::endl;  
    return 0;  
}
```

What is the exact output of this program?

Solution: 99 25 30 124 50

Question 3: Debugging [20 MARKS]

- (a) [4 MARKS] The following function is intended to find the maximum element in an array of integers. However, it contains a logical error that may lead to incorrect results or a crash. Identify the primary logical error and describe how to fix it.

```
int findMax(int arr[], int size) {  
    if (size == 0) {  
        return -1;  
    }  
    int maxValue = arr[0];  
    for (int i = 0; i <= size; ++i) {  
        if (arr[i] > maxValue) {  
            maxValue = arr[i];  
        }  
    }  
    return maxValue;  
}
```

Solution: The loop condition `i <= size` will cause the loop to attempt to access `arr[size]` on its last intended iteration. Array indices in range from `0` to `size - 1`. Accessing `arr[size]` is an out-of-bounds access. **Fix:** Change the loop condition to `i < size`.

- (b) [4 MARKS] The following function attempts to create and initialize a `Point` struct and return a pointer to it. However, it contains a critical error related to memory lifetime. Identify the error and describe how to fix it.

```
struct Point {  
    int x;
```



```
    int y;
};

Point* createPoint(int x, int y) {
    Point point;
    point.x = x;
    point.y = y;
    return &point;
}
```

Solution: The function `createPoint` returns the address of `localPoint`; a local variable that is stored on the stack and is destroyed when the function exits. The returned pointer therefore becomes a *dangling pointer*, pointing to invalid memory.

Fix: To return a pointer to a `Point` struct that persists after the function call, the `Point` should be allocated on the heap using **`new`**. The caller would then be responsible for releasing this memory using **`delete`**. A corrected implementation would be:

```
Point* createPoint(int x, int y) {
    Point* point = new Point; // Allocate on heap
    point->x = x;
    point->y = y;
    return point;
}
```

- (c) [12 MARKS] For each code snippet below, identify any syntax or logical errors. If there are no errors, write None. Provide a brief explanation for each identified error.

i.

```
int main() {
    int value = 10;
    int *ptr;
    *ptr = value;
    std::cout << *ptr;
    return 0;
}
```

Solution: Error: The pointer `ptr` is declared but not initialized. Dereferencing it in `*ptr = value;` to assign a value causes undefined behavior because `ptr` does not point to a valid memory location. **Fix (example):** Initialize `ptr` to point to `value`: `int *ptr = &value;` (Then `*ptr = value;` would be redundant, but `*ptr = 20;` would be valid). Or allocate memory: `ptr = new int;`.

ii.

```
std::string greet(std::string name) {  
    return "Hello " + name;  
}  
  
int main() {  
    std::cout << greet();  
    return 0;  
}
```

Solution: Error: The function call `greet()` in `main` is missing an argument. The function signature `std::string greet(std::string name)` requires one argument of type `std::string`.

iii.

```
int main() {  
    const int MAX_PLAYERS = 4;  
    MAX_PLAYERS = 5;  
    int scores[MAX_PLAYERS];  
    std::cout << scores[0];  
    return 0;  
}
```

Solution: Error: Attempting to modify a **const** variable `MAX_PLAYERS`. Once a **const** variable is initialized, its value cannot be changed. (Note: `scores[0]` is also uninitialized, which is a separate runtime issue of undefined behavior if read).

iv.

```
int main() {  
    int num = 7;  
    if (num = 0) {  
        std::cout << "Num is zero.";  
    }
```

```
    } else {  
        std::cout << "Num is not zero."  
    }  
    return 0;  
}
```

Solution: Logical Error (compiles, but likely not intended behavior): The condition `if (num = 0)` uses an assignment operator (`=`) not a comparison operator (`==`). This assigns `0` to `num`, and the expression `(num = 0)` evaluates to `0` (which is false in a boolean context). Thus, "Num is not zero." will be printed, which might be misleading.

v.

```
void displayFirst(int arrParam[]) {  
    std::cout << arrParam[0] << std::endl;  
}  
  
int main() {  
    int data[] = {100, 200, 300};  
    displayFirst(data);  
    return 0;  
}
```

Solution: None. The array `data` decays to a pointer when passed to `displayFirst`. Accessing `arrParam[0]` is valid and will print `100`.

vi.

```
struct Item {  
    std::string name;  
    double price;  
};  
  
int main() {  
    Item myItem;  
    myItem.name = "Book";  
    std::cout << "Price: " << myItem.price;  
    return 0;  
}
```

Solution: Error (Undefined Behavior): The member `myItem.price` is read without being initialized. Reading an uninitialized variable of a fundamental type like **double** leads to undefined behavior.

Question 4: Coding _____ **[35 MARKS]**

- (a) **[8 MARKS]** *Recursive Array Sum of Squares.* The following function recursively calculates the sum of the squares of elements in an integer array up to a given index. For example, if `arr = {1, 2, 3}`, `sumOfSquares(arr, 2)` should conceptually lead to $1^2 + 2^2 + 3^2 = 1 + 4 + 9 = 14$. The base case is when the index is less than 0. Complete the function by filling in the blanks.

```
long long sumOfSquares(const int* arr, int index) {
    if (index < ____ ) {
        return ____;
    }
    long long currentSquare = (long long)____ * ____;
    return currentSquare + ____;
}
```

Solution for blanks:

1. 0
 2. 0LL
 3. `arr[index]`
 4. `arr[index]`
 5. `sumOfSquares(arr, index - 1)`
- (b) **[5 MARKS]** *Vowel Shifter.* The function `shiftVowels` takes a `std::string` and shifts every vowel in it to the next vowel in the sequence `a e i o u a` (cyclically). For example, `a` becomes `e` and `u` becomes `a`. Non-vowel characters remain unchanged. Case should be preserved (e.g., `A` becomes `E`). Fill in the blanks.

```
#include <string>
#include <cctype>

char getNextVowel(char v, bool is_upper) {
    char next_v = v;
```

```

    switch (tolower(v)) {
        case 'a': next_v = 'e'; break;
        case 'e': next_v = 'i'; break;
        case 'i': next_v = 'o'; break;
        case 'o': next_v = 'u'; break;
        case 'u': next_v = 'a'; break;
    }
    return is_upper ? toupper(next_v) : next_v;
}

std::string shiftVowels(std::string s) {
    for (int i = 0; i < ____; ++i) {
        char currentChar = s[i];
        bool is_uppercase = isupper(currentChar);
        char lowerChar = tolower(currentChar);

        if (lowerChar == 'a' || lowerChar == 'e' || lowerChar == 'i' ||
            lowerChar == 'o' || ____ ) {
            s[i] = ____;
        }
    }
    return ____;
}

```

Solution for blanks:

1. s.length() (or s.size())
2. lowerChar == 'u'
3. getNextVowel(lowerChar, is_uppercase)
4. s

(c) [10 MARKS] *Find Product by ID.* You are given a structure Product and a std::vector of these products.

```

#include <string>
#include <vector>

struct Product {

```

```
    int productID;
    std::string productName;
    double price;
};
```

Write a function `Product* findProductById(const std::vector<Product>& products, int targetID)` that searches for a product with the given `targetID`. If found, it should return a pointer to a **dynamically allocated copy** of that `Product` struct. If not found, it should return `nullptr`.

Solution:

```
Product* findProductById(
    const std::vector<Product>& products,
    int targetID
) {
    for (const Product& prod : products) {
        if (prod.productID == targetID) {
            Product* foundProductCopy = new Product;
            foundProductCopy->productID = prod.productID;
            foundProductCopy->productName = prod.productName;
            foundProductCopy->price = prod.price;
            return foundProductCopy;
        }
    }
    return nullptr; // Not found
}
```

- (d) [12 MARKS] *Character Frequencies in a String*. Write a function that takes a string as input. The function should count the frequency of each alphabetic character (case-insensitive, i.e., a and A count towards the same character, a) that appears in the string. It should then print the frequencies of only those characters that appear at least once, in alphabetical order. For example, if the input is `Hello World!`, the output should be:

```
d: 1
e: 1
```

```
h: 1
l: 3
o: 2
r: 1
w: 1
```

(Non-alphabetic characters are ignored. Output is alphabetized by the lowercase character).

Solution:

```
#include <string>
#include <vector>
#include <iostream>
#include <cctype> // For tolower() and isalpha()

void printCharFrequencies(const std::string& text) {
    std::vector<int> frequencies(26, 0);

    for (char ch : text) {
        if (isalpha(ch)) {
            char lower_ch = tolower(ch);
            frequencies[lower_ch - 'a']++;
        }
    }

    for (int i = 0; i < 26; ++i) {
        if (frequencies[i] > 0) {
            char character = 'a' + i; // Convert index back to char
            std::cout << character << ": " << frequencies[i] << '\n';
        }
    }
}
```

[EXTRA SPACE]